



Container Orchestration Storage

Storage in Docker

 Copy page

This article covers Docker storage, file system management, layered architecture, and methods for persisting data using volumes and bind mounts.

Hello and welcome to this advanced Docker lesson focused on container storage and file system management. In this guide, we explore Docker storage drivers, layered architecture, and the key differences between persistent and ephemeral data management.

When Docker is installed, it creates a directory structure under `/var/lib/docker` with subdirectories such as `overlay2`, `containers`, `images`, and `volumes`. These directories hold data for images, running containers, and persistent volumes. For instance, container-related files are stored in the `containers` folder, image data in the `images` folder, and any persistent container data in the `volumes` folder.

Docker's Layered Architecture

Docker builds images using a layered structure, where every instruction in a Dockerfile creates a new layer containing only the changes relative to the previous one. Consider the following example Dockerfile:



>

```
RUN pip install flask flask-mysql
```

```
COPY . /opt/source-code
```

```
ENTRYPOINT ["flask", "run"]
```

Build the image with:

```
docker build -t mmumshad/my-custom-app .
```



In this example:

Base Layer: The official Ubuntu image.

Packages Layer: Installed APT packages.

Dependencies Layer: Python libraries installed via pip (Flask and flask-mysql).

Application Code Layer: Your source code copied into the image.

Entry Point Layer: The defined startup command.

Because layers only store changes from previous ones, Docker determines the overall image size based on these incremental modifications. For example, the base Ubuntu image might be around 120 MB, while the code and configuration layers remain relatively small.

Reusing Layers Across Applications

Imagine a scenario with two applications that share most of the Dockerfile layers. Even if the source code and entry points differ, they use the same base image and dependency layers, ensuring efficient caching and faster builds.

Dockerfile for the first application:



>

```
RUN pip install flask flask-mysql
```

```
COPY . /opt/source-code
```

```
ENTRYPOINT ["flask", "run"]
```

Build with:

```
docker build -t mmumshad/my-custom-app .
```



Dockerfile2 for the second application:

```
FROM ubuntu
```



```
RUN apt-get update && apt-get -y install python
```

```
RUN pip install flask flask-mysql
```

```
COPY app2.py /opt/source-code
```

```
ENTRYPOINT ["flask", "run"]
```

Build with:

```
docker build -t mmumshad/my-custom-app-2 -f Dockerfile2 .
```



Since the first three layers are identical, Docker reuses the cached layers and only rebuilds the layers that differ (i.e., the source code copy and the entry point configuration). This efficient caching mechanism speeds up your builds and minimizes disk usage.

>

Understanding Image and Container Layers

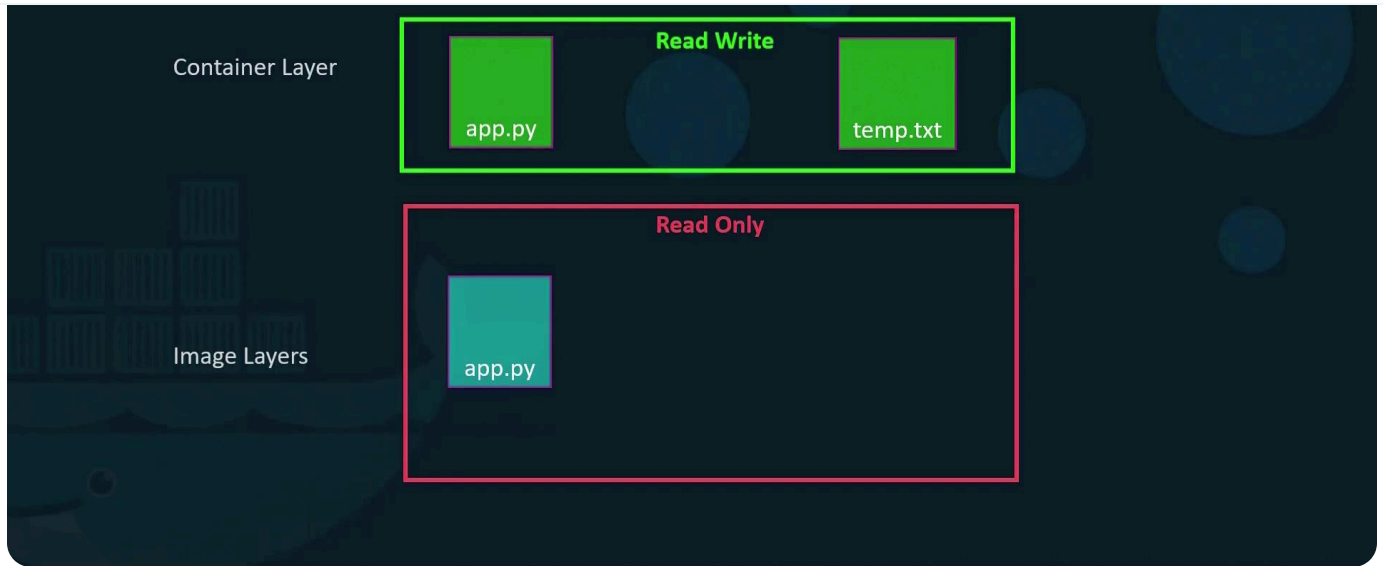
Docker images are composed of several immutable layers, stacked in the following order:

1. **Base Layer:** The underlying Ubuntu OS.
2. **Packages Layer:** OS-level packages installed via APT.
3. **Dependencies Layer:** Python packages installed with pip.
4. **Application Code Layer:** Your application source code.
5. **Entry Point Layer:** The command configuration for running the container.

When a container is launched, Docker appends a writable layer on top of these read-only layers. This writable layer captures runtime changes like log files, temporary data, or user modifications. For example, if you create a file named `temp.txt` within a running container, it is stored in this writable layer. Docker's "copy-on-write" mechanism ensures that modifications to files originally part of the image are first copied to the writable layer before any changes are applied.

The diagram below illustrates the "Copy-On-Write" concept:

>



Persisting Data with Volumes and Bind Mounts

Since the writable layer is temporary, persisting important data—especially for stateful applications like databases—is critical. Docker offers two primary methods for data persistence: volumes and bind mounts.

Using Volumes

Volumes are managed by Docker and provide a straightforward way to persist data. Create a volume on the Docker host with:

```
docker volume create data_volume
```



This command creates a directory under `/var/lib/docker/volumes/data_volume`. To mount the volume into a container (e.g., for MySQL data persistence), run:

```
docker run -v data_volume:/var/lib/mysql mysql
```





>

Using Bind Mounts

Bind mounts allow you to use an existing directory on your host system. For example, to use `/data/mysql`, run:

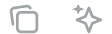
```
docker run -v /data/mysql:/var/lib/mysql mysql
```



Using the `--mount` Option

The `--mount` flag provides a more verbose syntax but achieves the same result. For a bind mount, use:

```
docker run \  
  --mount type=bind,source=/data/mysql,target=/var/lib/mysql \  
  mysql
```



Both the `-v` flag and the `--mount` option enable you to map a host directory to a container, ensuring your important data persists beyond the container's life span.

Docker Storage Drivers

Docker storage drivers are key to implementing the layered filesystem and managing the writable container layer. They handle the creation of layers and the copy-on-write mechanism. Common storage drivers include:

AUFS

ZFS

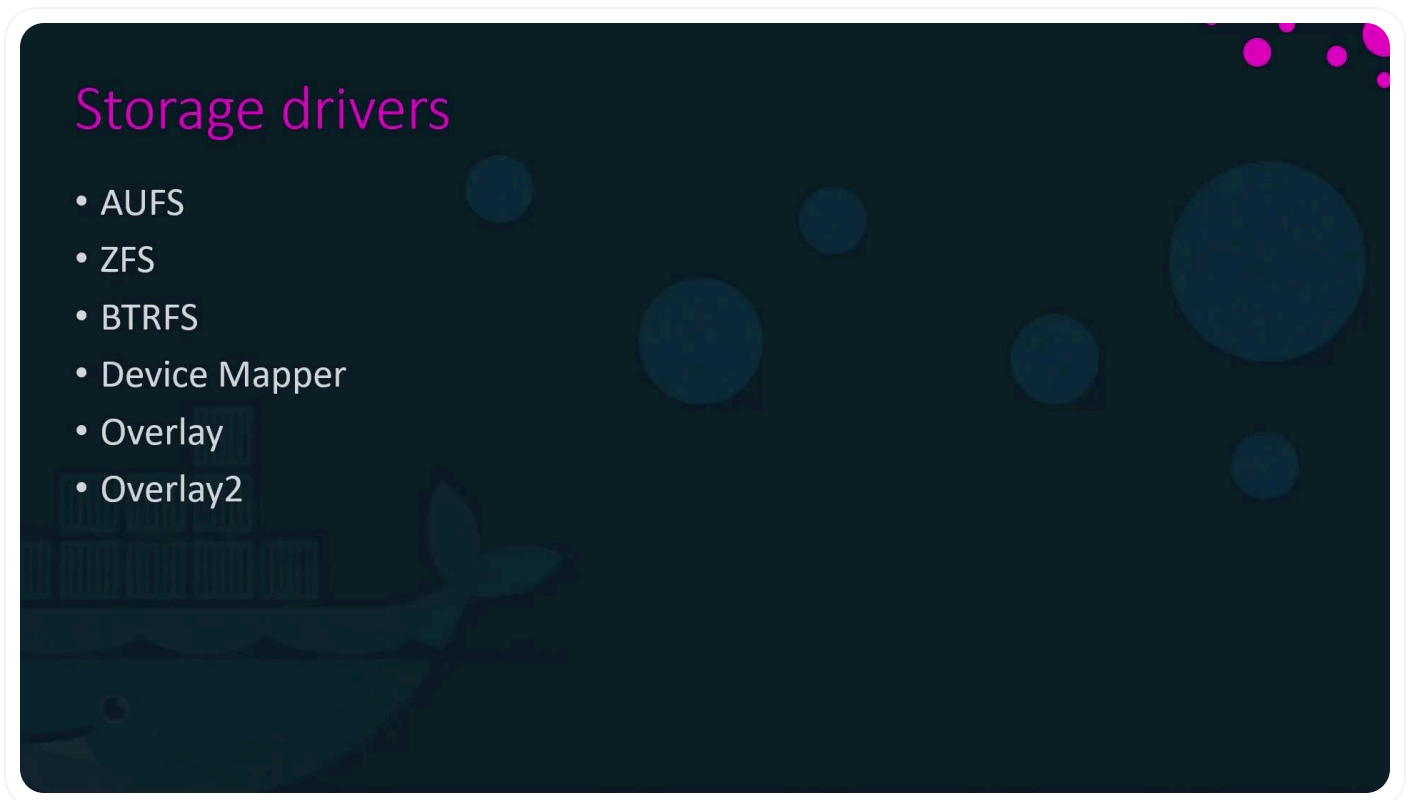
Btrfs

>

typically uses AUFS, while Fedora or CentOS might use Device Mapper. Docker automatically picks the optimal storage driver based on your system, but each driver offers unique performance and stability characteristics.

For a deeper understanding of Docker storage drivers and their use cases, consult the official [Docker Documentation](#) and other relevant resources.

The diagram below showcases various storage drivers, including AUFS, ZFS, BTRFS, Device Mapper, Overlay, and Overlay2:



This concludes our exploration of Docker storage concepts, including layered architecture, the copy-on-write mechanism, persistent data storage with volumes and bind mounts, and the role of storage drivers.

For more related topics, check out these resources:



>



Watch Video

[Learn more >](#)

Was this page helpful?



Yes



No

Introduction to Docker Storage

[< Previous](#)

Volume Driver Plugins in Docker

[Next >](#)

Powered by [mintlify](#)